

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name & Reg. No.	Sadhumitha S.	24011101091

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network (CNN) is a deep neural network model specifically designed for grid-structured inputs like images. While a vanilla feedforward neural network treats each pixel as an individual input, a CNN learns through filters that scan the image to detect local features like edges and corners.

A CNN consists of

- Convolution Layer – detects local features using learnable filters.
- Activation Layer – adds non-linearity (e.g., ReLU activation function).
- Pooling Layer – reduces the dimensionality of the output feature maps.
- Fully Connected Layer – aggregates the learned features and makes predictions.

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size
 F Filter Size
 P Padding
 S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

The ten classes present in the dataset are Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, and Truck. The dataset was loaded directly using `tensorflow.keras.datasets.cifar10`, and the pixel values were normalized to the range $[0, 1]$ before being fed into the network.

8. Experimental Procedure

Task 1: Loading and Exploring the Dataset

The CIFAR-10 dataset was loaded and split into training and testing sets. Ten sample images were displayed along with their class labels, and the class distribution across the training set was plotted to check whether the dataset is balanced.



Figure 1: Sample images from the CIFAR-10 dataset with their class labels.

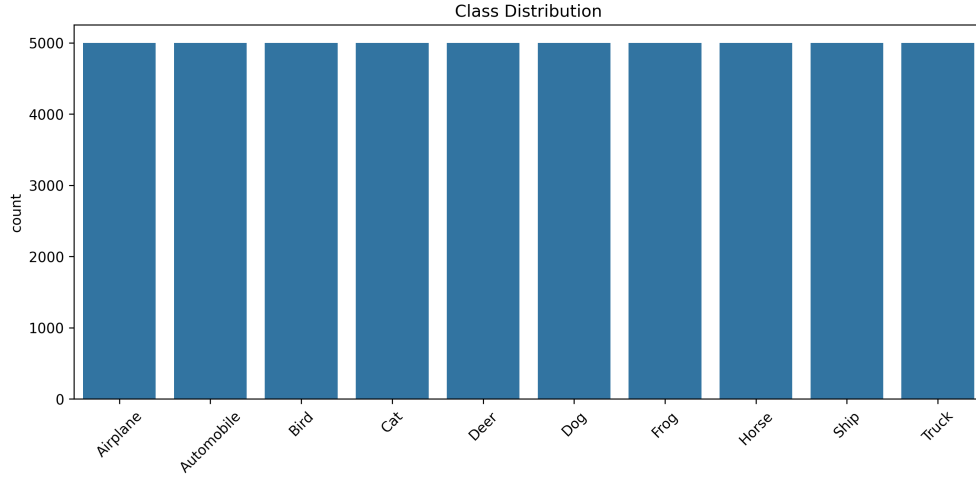


Figure 2: Class distribution of the CIFAR-10 training set.

Task 2: Effect of Kernel Size

A convolution layer was implemented and tested with three different kernel sizes: 3×3 , 5×5 , and 7×7 , keeping the stride and padding fixed. The resulting feature map sizes were recorded for each case.

Table 1: Feature map size for different kernel sizes (Input = 32×32 , Stride = 1, Padding = Valid)

Kernel Size	Formula	Output Size
3×3	$(32 - 3)/1 + 1$	30×30
5×5	$(32 - 5)/1 + 1$	28×28
7×7	$(32 - 7)/1 + 1$	26×26

Inference: As the kernel size increases, the output feature map shrinks faster. A larger kernel captures more spatial context but loses fine spatial detail.

Task 3: Effect of Stride and Padding

The effect of stride (1 and 2) and padding (**same** and **valid**) on the output feature map dimensions was studied using a fixed kernel size.

Table 2: Effect of stride and padding on output dimensions

Stride	Padding	Formula	Output Size
1	Same	N	32×32
1	Valid	$(N - F)/1 + 1$	30×30
2	Same	$\lceil N/S \rceil$	16×16
2	Valid	$(N - F)/2 + 1$	15×15

Inference: **Same** padding keeps the spatial size unchanged by adding zero borders, while **valid** padding shrinks it. A stride of 2 roughly halves the feature map dimensions, reducing computation.

Task 4: Feature Map Visualization

The output of the first convolution layer was extracted for a sample input image, and eight feature maps (out of the total number of filters in that layer) were plotted to visualize what patterns the network is picking up right after the first layer.

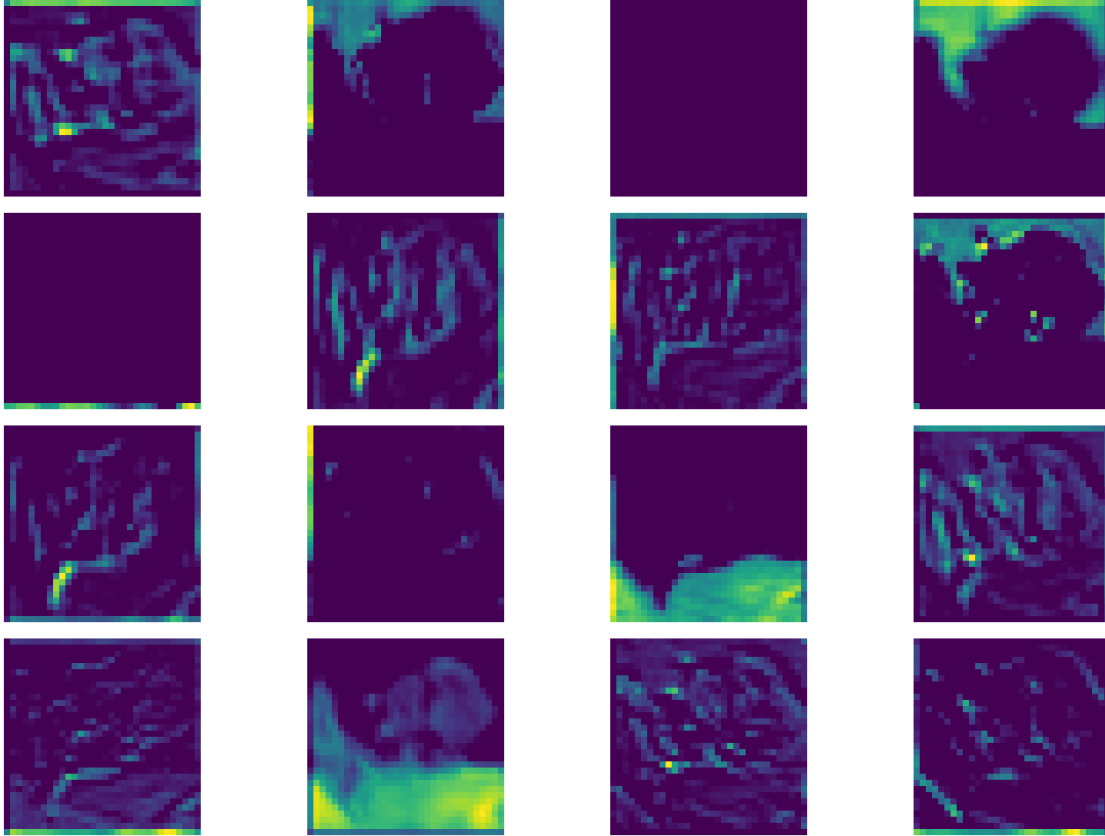


Figure 3: Feature maps generated by the first convolution layer.

Inference: The feature maps highlight different low-level features such as edges, colour blobs, and textures. Different filters in the first layer respond uniquely to various parts of the input image.

Task 5: Max Pooling vs Average Pooling

Two versions of the same CNN were trained – one using Max Pooling and the other using Average Pooling – and their output sizes and classification accuracies were compared.

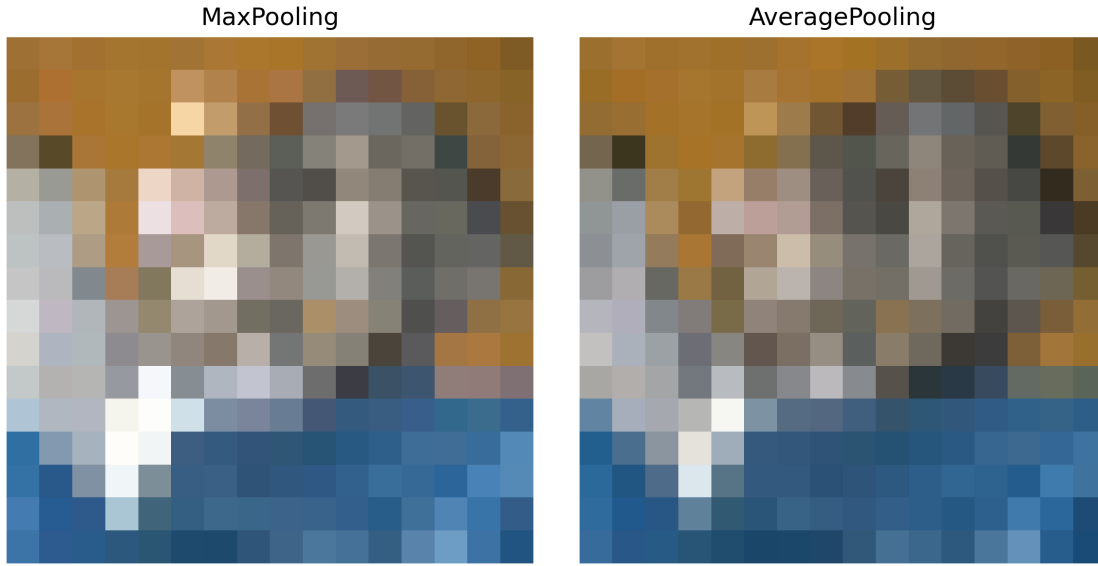


Figure 4: Comparison of Max Pooling and Average Pooling.

Table 3: Max Pooling vs Average Pooling

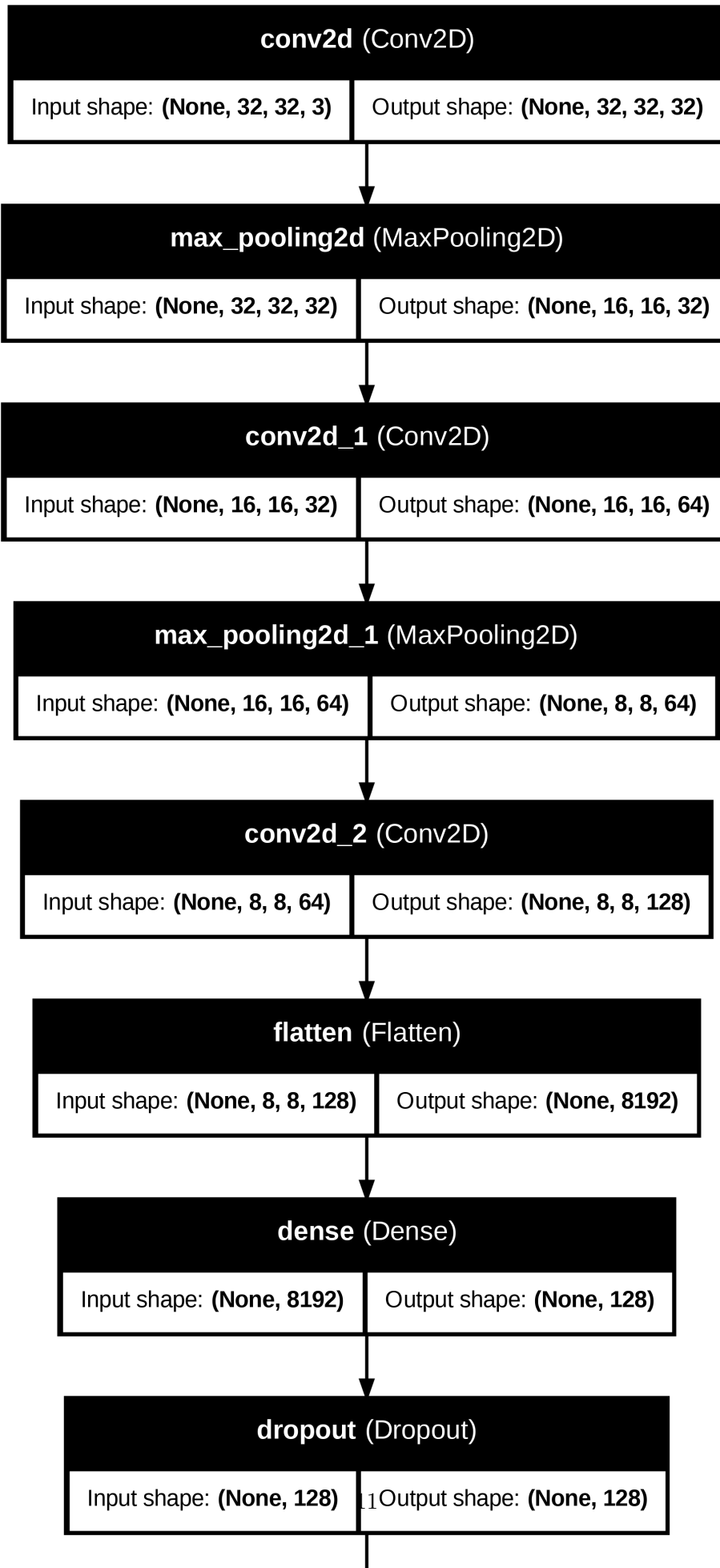
Metric	Max Pooling
Output Feature Map Size	16×16
Test Accuracy (%)	74.10

Inference: Max pooling preserves the most prominent features resulting in sharper maps, while average pooling smooths the feature map. The model using max pooling achieved a test accuracy of 74.10%.

Task 6: Building and Training the CNN

The following CNN architecture was constructed and trained on the CIFAR-10 dataset:

Input ($32 \times 32 \times 3$) $\rightarrow$ Conv2D(32, ReLU) $\rightarrow$ MaxPool
 $\rightarrow$ Conv2D(64, ReLU) $\rightarrow$ MaxPool
 $\rightarrow$ Conv2D(128, ReLU) $\rightarrow$ Flatten
 $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout
 $\rightarrow$ Dense(10, Softmax)



Training configuration:

- Optimizer : Adam
- Loss Function : Categorical Cross-Entropy
- Epochs : 20
- Batch Size : 32

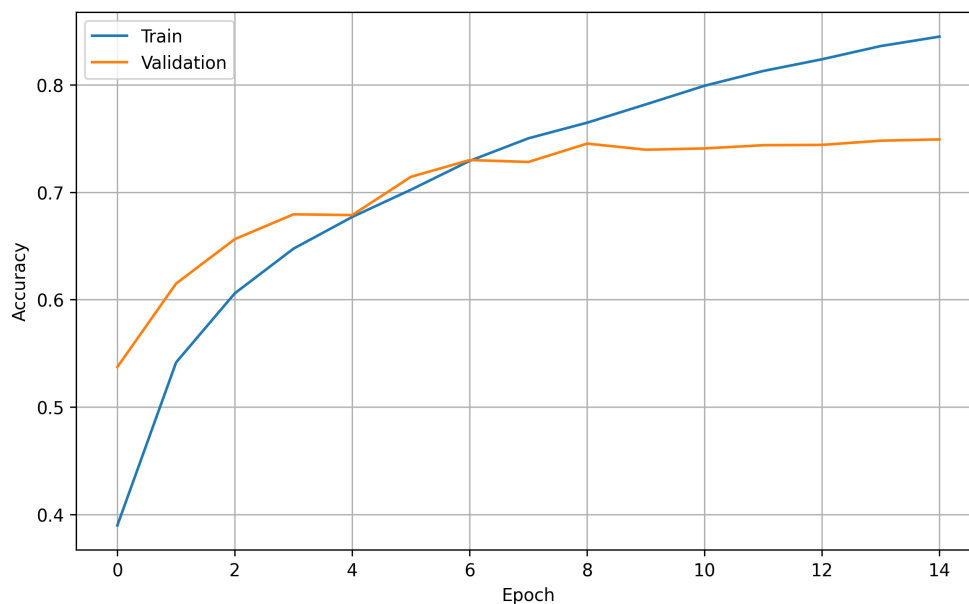


Figure 6: Training and validation accuracy over epochs.

Inference: The training accuracy increased steadily, reaching 84.47%, while the validation accuracy plateaued around 74.91%. The curves diverged slightly in the later epochs, indicating a small amount of overfitting.

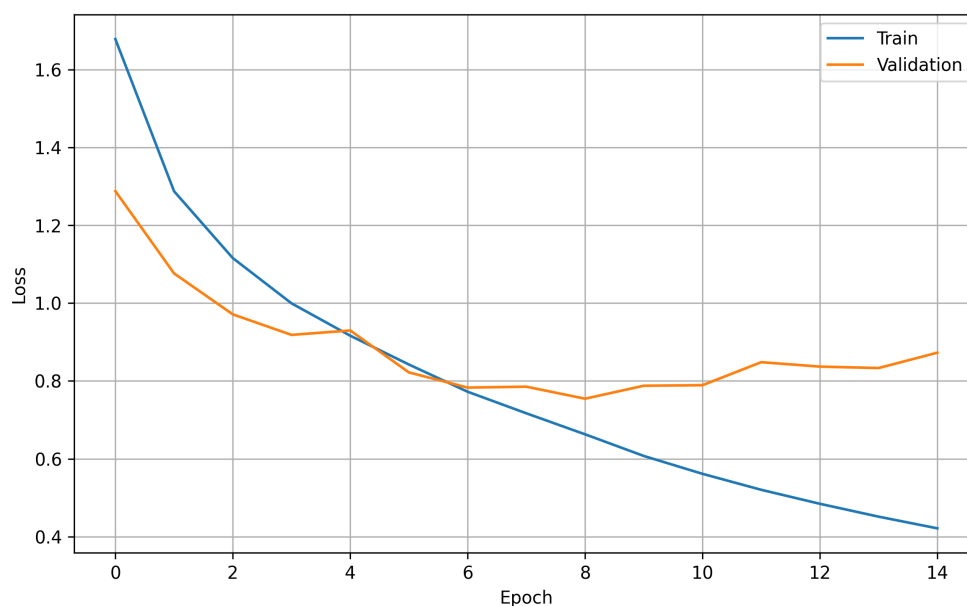


Figure 7: Training and validation loss over epochs.

Inference: The training loss decreased smoothly to 0.4214, but the validation loss started increasing

slightly after epoch 9 (reaching 0.8727), confirming the presence of mild overfitting.

Task 7: Model Evaluation

The trained model was evaluated on the CIFAR-10 test set using accuracy, precision, recall, F1-score, the confusion matrix, and the classification report.

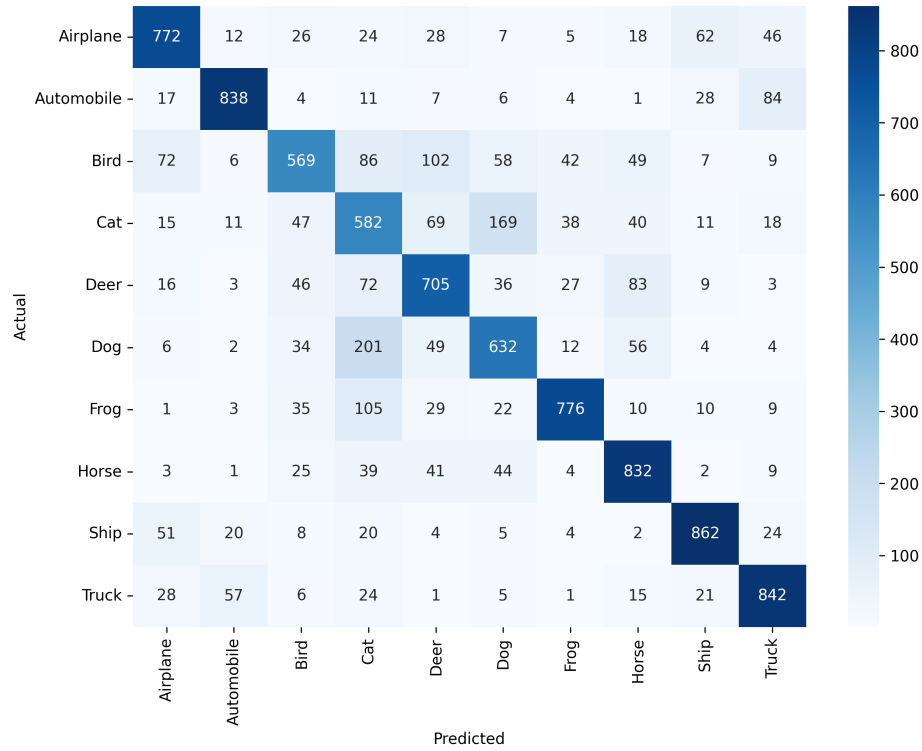


Figure 8: Confusion matrix of the trained CNN on the CIFAR-10 test set.

Inference: The model classified Automobile and Ship most accurately. It most frequently confused visually similar classes, such as Cats and Dogs, as well as Birds and Deer.

9. Mandatory Plots

All eight required plots are included with the corresponding tasks above. For quick reference:

1. Sample Images – Figure 1
2. Class Distribution – Figure 2
3. Training Accuracy – Figure 6
4. Validation Accuracy – included in Figure 6
5. Training Loss – Figure 7
6. Validation Loss – included in Figure 7
7. Feature Maps – Figure 3
8. Confusion Matrix – Figure 8

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

11. Results

Table 4: Final results of the CNN model on CIFAR-10

Metric	Value
Training Accuracy (%)	84.47
Testing Accuracy (%)	74.10
Precision	0.75
Recall	0.74
F1-score	0.74
Number of Parameters	1,143,242

12. Source Code

[GitHub Repository](#)

13. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?

Fully connected layers take each pixel individually and have a large number of parameters. Convolution uses small filters that traverse the entire image and the weights are shared, hence it decreases the number of parameters. Moreover, it maintains spatial information of the image.

2. How does stride affect the feature map size?

Stride is the step size that decides how far the filter moves every time. Stride 1 gives larger feature maps whereas a stride of 2 results in smaller feature maps with less computation.

3. What is the role of padding?

Padding means adding pixels to the edges of an image before applying the convolution operation. It is done so that the feature maps are not decreased too fast in size and pixels at borders are used correctly. Same padding gives an output with the same spatial size as the input.

4. Why is pooling used?

Pooling decreases the spatial size of the feature maps, which in turn reduces computation, and the model becomes robust to the locations of the features.

5. How do feature maps represent image characteristics?

A feature map shows where a particular pattern is detected in an image. Early CNN layers usually detect simple features such as edges and colours, while deeper layers combine them to recognize more complex shapes and object parts.

6. Why do CNNs require fewer parameters than MLPs?

CNNs use small filters and reuse the same weights across the image. Each neuron also only looks at a small local region. MLPs, on the other hand, connect every input pixel to every neuron, which results in many more parameters.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.